Hướng dẫn dự án Quản lý đồ án

Cấu trúc, luồng hoạt động, cách chạy và công nghệ sử dụng

Tài liệu này hướng dẫn cách đọc và khởi chạy ứng dụng Flutter Android cùng ASP.NET Core Web API và SQL Server. API và app Android đã chạy được với SQL Server. Database QuanLyDoAn đã được tạo từ script; trang chủ, danh sách lọc và trang chi tiết đều tải được dữ liệu mẫu.

Trạng thái hiện tại

Connection string trỏ tới instance LAPTOP-VHGPK0SP bằng Windows Authentication. Database QuanLyDoAn đã được tạo và nạp dữ liệu mẫu; Swagger, các API dữ liệu, danh sách lọc và trang chi tiết đã chạy trên Android.

Kiến trúc tổng thể

Lớp	Thành phần	Vai trò
Giao diện Android	Flutter / Dart	Hiển thị trang chủ, danh sách, chi tiết và trạng thái đang tải/lỗi.
API	ASP.NET Core 8	Nhận HTTP GET, truy vấn dữ liệu và trả JSON cho ứng dụng.
Truy cập dữ liệu	Entity Framework Core 8	Ánh xạ bảng SQL Server thành entity và tạo truy vấn.
Lưu trữ	SQL Server	Lưu loại đề tài, chuyên ngành, đề tài và ngành học.

Luồng dữ liệu: Người dùng → trang Flutter → ApiService → HTTP GET → Controller → QuanLyDoAnContext → SQL Server. Kết quả đi ngược lại dưới dạng DTO/JSON, được chuyển thành model Dart rồi hiển thị trên màn hình.

Cấu trúc mã nguồn

Đường dẫn	Trách nhiệm chính
FlutterBaiTap02/lib/main.dart	Khởi tạo MaterialApp, theme và HomePage.
FlutterBaiTap02/lib/pages/	Các màn hình chọn loại/chuyên ngành, danh sách và chi tiết đề tài.
FlutterBaiTap02/lib/services/api_service.dart	Tạo URL, gửi GET, giải mã JSON, đặt timeout và chuẩn hóa lỗi.
FlutterBaiTap02/lib/models/	Chuyển JSON từ API thành các đối tượng Dart.
FlutterBaiTap02/lib/theme/ và widgets/	Màu sắc, kiểu chữ, app bar và trạng thái dùng chung.
QuanLyDoAnApi/Controllers/	Các endpoint HTTP cho loại đề tài, chuyên ngành, đề tài, ngành học.
QuanLyDoAnApi/Data, DTOs, Entities/	Cấu hình EF Core, kiểu dữ liệu API và entity ánh xạ SQL.
QuanLyDoAn.sql	Tạo database/bảng, khóa, chỉ mục và dữ liệu mẫu.

Luồng sử dụng trên Android

1. Khi mở app, main.dart tạo MaterialApp với theme chung và nạp HomePage.
2. HomePage gọi song song API loại đề tài và chuyên ngành bằng Future.wait; dữ liệu được sắp xếp theo thứ tự mã phù hợp giao diện.
3. Chọn loại đề tài hoặc chuyên ngành sẽ mở ProjectListPage với mã bộ lọc tương ứng.
4. ProjectListPage gửi GET /api/de-tai kèm maLoai và/hoặc maCN. Khi người dùng đổi bộ lọc nhanh, _requestVersion bỏ qua kết quả của request cũ.
5. Chạm một đề tài sẽ mở ProjectDetailPage và gọi GET /api/de-tai/{maDeTai}.
6. ApiService chuyển JSON thành model; Flutter cập nhật giao diện ở trạng thái dữ liệu, rỗng, đang tải hoặc lỗi. Lỗi có nút thử lại.

Các điểm xử lý bất đồng bộ

Các trang kiểm tra mounted trước khi gọi setState sau await để tránh cập nhật widget đã bị đóng. ApiService đặt timeout 15 giây. Những điểm này giúp app không treo vô hạn và giảm lỗi khi chuyển trang trong lúc request đang chạy.

Web API và cơ sở dữ liệu

Program.cs đọc ConnectionStrings:QuanLyDoAn, đăng ký QuanLyDoAnContext với SQL Server, cấu hình JSON camelCase và bật Swagger trong Development. UseExceptionHandler trả lỗi theo ProblemDetails; chi tiết SQL không được gửi trực tiếp cho client.

Controller truy vấn EF Core theo từng chức năng. Các controller hiện cung cấp endpoint GET để đọc dữ liệu. DeTaiController hỗ trợ lọc theo loại/chuyên ngành và dùng LEFT JOIN với LoaiDeTai để vẫn lấy đề tài có MaLoai = NULL.

Endpoint	Ý nghĩa
GET /api/loai-de-tai	Danh sách loại đề tài.
GET /api/chuyen-nganh	Danh sách chuyên ngành.
GET /api/de-tai	Danh sách đề tài; có thể thêm maLoai và maCN.
GET /api/de-tai/{maDeTai}	Chi tiết một đề tài theo mã.
GET /api/nganh-hoc	Danh sách ngành học.
GET /api/nganh-hoc/{maNganh}	Chi tiết một ngành học theo mã.

EF Core và SQL Server

QuanLyDoAnContext ánh xạ entity vào các bảng dbo.LoaiDeTai, dbo.ChuyenNganh, dbo.DeTai và dbo.NganhHoc. Controller dùng AsNoTracking cho truy vấn chỉ đọc và chiếu dữ liệu sang DTO thay vì trả trực tiếp entity EF.

QuanLyDoAn.sql tạo database nếu chưa có, tạo bảng có điều kiện, khai báo khóa ngoại/chỉ mục và thêm dữ liệu mẫu trong transaction. Script không chứa DROP DATABASE, DROP TABLE, TRUNCATE hoặc DELETE. Nó có thể cập nhật mô tả chuyên ngành khi giá trị hiện tại còn NULL.

Schema đã xác nhận

Database QuanLyDoAn đã được tạo từ QuanLyDoAn.sql trên instance LAPTOP-VHGPK0SP. Dữ liệu mẫu và khóa ngoại đã được kiểm tra; DBCC CHECKCONSTRAINTS không báo lỗi, và các endpoint đọc dữ liệu hiện trả HTTP 200.

Yêu cầu để chạy dự án

Công cụ	Phiên bản hoặc cấu hình
.NET SDK	Project API target net8.0; máy kiểm tra có .NET SDK 10.0.400.
Flutter và Dart	Máy kiểm tra: Flutter 3.47.7, Dart 3.13.5; pubspec yêu cầu Dart ^3.13.5.
Android SDK và emulator	Android Studio/SDK, ADB và một AVD; lần chạy dùng Week7Bai08Api34_Codex.
Java	Máy kiểm tra có OpenJDK 21.0.12; cấu hình Android đặt source/target JVM 17.
SQL Server	Default instance LAPTOP-VHGPK0SP, Windows Authentication; Windows login chạy API cần quyền đọc database.

Bước 1 Chuẩn bị SQL Server

1. Nếu SQL Server đang dừng, mở Services bằng quyền quản trị, tìm SQL Server (MSSQLSERVER) và chọn Start.
2. Trong SSMS, kết nối LAPTOP-VHGPK0SP bằng Windows Authentication. Nếu database QuanLyDoAn chưa có, mở QuanLyDoAn.sql và chạy trên đúng instance. Xác nhận các bảng và dữ liệu mẫu đã được tạo.
3. Đảm bảo Windows account chạy API có quyền kết nối và đọc database QuanLyDoAn.

Bước 2 Chạy ASP.NET Core API

Cấu hình Development hiện dùng instance đã chọn:

```
Server=LAPTOP-VHGPK0SP;Database=QuanLyDoAn;Trusted_Connection=True;TrustServerCertificate=True;
```

PowerShell

```
cd "E:\Learn\Ky5_26_27\BaiTapTuan10\QuanLyDoAnApi"
dotnet restore
dotnet build
dotnet run --launch-profile http
```

Profile http đặt môi trường Development và lắng nghe tại http://localhost:5130. Mở http://localhost:5130/swagger để xác nhận API đã khởi động.

Bước 3 Chạy Flutter trên Android emulator

PowerShell

```
cd "E:\Learn\Ky5_26_27\BaiTapTuan10\FlutterBaiTap02"
flutter pub get
flutter emulators --launch Week7Bai08Api34_Codex
flutter devices
flutter run -d emulator-5554 --dart-define=API_BASE_URL=http://10.0.2.2:5130
```

Trong Android emulator, 10.0.2.2 trỏ về máy Windows host. Không dùng localhost làm API_BASE_URL trong emulator vì localhost khi đó là chính thiết bị ảo.

Có thể đổi địa chỉ API khi chạy Flutter bằng --dart-define=API_BASE_URL=...; ApiService đọc giá trị này. Mặc định trong source cũng là http://10.0.2.2:5130.

Thư viện và công nghệ chính

Công nghệ	Dùng ở đâu
Flutter / Dart	Xây dựng giao diện đa nền tảng và quản lý trạng thái trang.
http	Gửi HTTP GET đến ASP.NET Core API.
flutter_svg	Hiển thị các biểu tượng SVG lấy từ assets/figma/.
ASP.NET Core 8	Tạo Web API, route/controller, Swagger và middleware lỗi.
EF Core SqlServer	Truy vấn SQL Server qua entity/DbContext.
SQL Server	Lưu dữ liệu nghiệp vụ và quan hệ bảng.
Gradle / Android SDK / ADB	Build, cài đặt và chạy ứng dụng Flutter trên emulator.

Nếu cần mở APK đã có

Lần build mới từ source ở phiên trước dừng tại Gradle trước khi biên dịch Dart vì Java báo Unable to establish loopback connection. APK debug có sẵn đã được cài và mở trên emulator; lần chạy hiện tại đã xác nhận giao diện và request tới API hoạt động.

```
cd "E:\Learn\Ky5_26_27\BaiTapTuan10\FlutterBaiTap02"
adb install -r build\app\outputs\flutter-apk\app-debug.apk
adb shell monkey -p com.example.flutter_bai_tap_02 1
```

Kết quả review và kiểm chứng

Hạng mục	Kết quả lần kiểm tra
Build API	Thành công: 0 warning, 0 error.
flutter analyze	Thành công: No issues found.
Swagger	GET /swagger/v1/swagger.json trả HTTP 200.
API đọc dữ liệu	Sáu endpoint được kiểm tra đều trả HTTP 200 sau khi database được tạo.
Ứng dụng Android	Trang chủ tải được dữ liệu; lọc theo chuyên ngành và xem chi tiết đề tài thành công.
Build Android mới	Lần build APK mới từ source ở phiên trước dừng trong Gradle với lỗi Unable to establish loopback connection; APK có sẵn vẫn chạy và đã kiểm tra luồng dữ liệu.

Nhận xét chính

- Đã khôi phục luồng dữ liệu: SQL Server đã khởi động, QuanLyDoAn được tạo và nạp dữ liệu mẫu gồm 4 loại đề tài, 5 chuyên ngành, 10 đề tài và 2 ngành học. Các endpoint hiện trả HTTP 200.
- Chẩn đoán lỗi phía Flutter còn hạn chế: ApiService chỉ giữ status code khi HTTP không thành công, chưa đọc title/detail an toàn từ ProblemDetails. Vì vậy người dùng thấy HTTP 500 thay vì thông tin dễ xử lý hơn.
- Các mapping EF Core đã chạy được trên những endpoint hiện tại. Nếu schema thay đổi, hãy scaffold hoặc so sánh lại cấu hình với database.
- API hiện là luồng đọc dữ liệu: các route được triển khai là GET. Màn hình hiện tập trung chọn/lọc và xem chi tiết đề tài.

Cách kiểm tra sau khi SQL Server chạy

1. Mở Swagger và gọi GET /api/chuyen-nganh; kết quả mong đợi là HTTP 200 cùng danh sách JSON.
2. Mở app Android; trang chủ phải hiện loại đề tài và chuyên ngành.
3. Chọn một chuyên ngành hoặc loại đề tài; danh sách phải thay đổi theo query maCN/maLoai.
4. Chạm một đề tài; trang chi tiết phải hiện mã, tên, số lượng tối đa, chuyên ngành, giảng viên và yêu cầu.
5. Tắt API hoặc nhập mã không hợp lệ để kiểm tra trạng thái lỗi, nút thử lại và trạng thái rỗng.

Tóm tắt để trình bày

Flutter chịu trách nhiệm giao diện và gửi request. ASP.NET Core nhận request, EF Core truy vấn SQL Server, DTO được chuyển thành JSON. Android emulator gọi máy host qua 10.0.2.2. Lần kiểm tra xác nhận build/analyze, API, dữ liệu SQL Server, lọc đề tài và trang chi tiết trên Android.